

Reducibility

Md. Rifat Rahman

June 2026

1 Introduction

Let's start with an example to get a feel for what reducibility means. Say you have two positive integers a and b , and you want to find their lcm. There are a few ways to go about this. One option is to get there through the gcd, using a formula you already know:

$$a \times b = \gcd(a, b) \times \text{lcm}(a, b).$$

So if you already have the gcd of those two numbers, you can just plug it into the formula and read off the lcm. What you're really doing here is *converting* a problem about finding an lcm into a problem about finding a gcd.

So your algorithm ends up working like this:

1. Take two positive integers a, b as input.
2. Compute $\gcd(a, b)$.
3. Output $\text{lcm}(a, b) = \frac{a \times b}{\gcd(a, b)}$.

That example is exactly what a *reduction* is: you solve one problem by turning it into another problem you already know how to solve.

2 General Discussion

Let's now build up the idea a bit more. Say you're handed a problem A that nobody has looked at before. You poke at it for a while and notice something useful: with a few changes, an input for A can be turned into an input for a problem B that everybody already knows how to solve. What's your strategy?

1. Take the input of problem A .
2. Make changes to it so that it becomes an input of problem B .
3. Run the algorithm for B .
4. Turn the output of B back into an output for A , and return it.

In our first example, A was finding the lcm and B was finding the gcd. When we can do this, we say A *reduces to* B , written $A \leq B$. The thing to take away is that A is *no harder* than B — because if you hand me a solution for B , I can build one for A out of it.

Using reduction to prove impossibility

Now here's a different situation. You're given a problem C , and this time no amount of poking gets you anywhere. You start to suspect the thing just can't be solved at all. But how do you actually *prove* that? You can't just say "I tried for months and got nothing." That's not an argument — somebody smarter might walk in tomorrow with the clever trick you missed. This is where reduction earns its keep. Suppose D is some problem that everybody already agrees is unsolvable. Here's what you do:

1. Take any input for problem D .
2. Convert it into an input for problem C , arranged so that the answer for C tells you the answer for D .

Now look at what you've got. If C were solvable, you could run your solution for C and read off the answer to D . But D is unsolvable, so that's a contradiction. C must be unsolvable too. Somebody might ask: what if we go the other way, taking an instance of C and turning it into the unsolvable D ? Does the proof still work? No, and it's worth seeing why. We said earlier that when A reduces to B , A is no harder than B . So reducing C to D would only tell us that C is no harder than something unsolvable — which tells us nothing at all. Plenty of easy problems are no harder than an unsolvable one.

Going the right way — unsolvable D *into* the unknown C — proves that C is at least as hard as D . And since D is already unsolvable, so is C . The direction really matters here, so it's worth pausing on it.

So, to summarise:

1. If you want to prove a problem is **solvable**, convert *it* into a known solvable problem.
2. If you want to prove a problem is **unsolvable**, convert a known *unsolvable* problem *into it*.

3 Proving a Problem is Decidable

To show a language is decidable, we go in the easy direction: reduce it to some language we already know is decidable. Recall from the previous note that

$$A_{\text{DFA}} = \{ \langle D, w \rangle \mid D \text{ is a DFA that accepts the input string } w \}$$

is decidable, and here's the machine that decides it:

M_{DFA} = "On input $\langle D, w \rangle$, where D is a DFA and w is a string:

1. Simulate D on input w .
2. If the simulation ends in an accept state, *accept*; if it ends in a non-accepting state, *reject*."

The simulation always finishes after $|w|$ steps, so M_{DFA} never loops — it's a decider.

We'll now treat A_{DFA} as our trusted, already-solved problem, and reduce the other two membership problems to it.

Theorem 1. $A_{\text{NFA}} = \{ \langle N, w \rangle \mid N \text{ is an NFA that accepts the input string } w \}$ is decidable.

Proof. Here's a machine M_{NFA} that decides A_{NFA} .

M_{NFA} = "On input $\langle N, w \rangle$, where N is an NFA and w is a string:

1. Convert the NFA N into an equivalent DFA D using the subset construction.

2. Run M_{DFA} on input $\langle D, w \rangle$.
3. If M_{DFA} accepts, *accept*; otherwise *reject*.”

Why does this halt? Step 1 finishes because the subset construction is a finite procedure — it spits out a DFA with at most $2^{|Q|}$ states, where Q is the state set of N . It might be a big DFA, but it’s a finite one, and we get it in finite time. Step 2 finishes because M_{DFA} is a decider. So M_{NFA} halts on every input.

And why is the answer right? Because the subset construction gives us $L(D) = L(N)$ — the two machines accept exactly the same strings. So $\langle N, w \rangle \in A_{\text{NFA}}$ exactly when $w \in L(N)$, which is exactly when $w \in L(D)$, which is exactly when $\langle D, w \rangle \in A_{\text{DFA}}$. That last one is what M_{DFA} checks for us. $\square$

Theorem 2. $A_{\text{REX}} = \{ \langle R, w \rangle \mid R \text{ is a regular expression that generates the string } w \}$ is decidable.

Proof. Here’s a machine M_{REX} that decides A_{REX} .

M_{REX} = “On input $\langle R, w \rangle$, where R is a regular expression and w is a string:

1. Convert R into an equivalent NFA N using Thompson’s construction.
2. Run M_{NFA} on input $\langle N, w \rangle$.
3. If M_{NFA} accepts, *accept*; otherwise *reject*.”

Same two questions as before. Thompson’s construction always terminates, giving us an NFA whose size is linear in the length of R , and M_{NFA} is a decider by the theorem we just proved — so M_{REX} halts on every input. As for correctness, Thompson’s construction guarantees $L(N) = L(R)$, so $\langle R, w \rangle \in A_{\text{REX}}$ exactly when $\langle N, w \rangle \in A_{\text{NFA}}$. $\square$

Notice both proofs have the same shape. Each one takes the unknown problem and converts it into something we already know is decidable — the easy direction. And in both cases we had to check two things: that the conversion itself finishes, and that it preserves the answer. Skip either check and the proof has a hole in it, so get in the habit of asking both questions every time.

4 Proving a Problem is Undecidable

Using reducibility to show a problem is undecidable is the more interesting direction. As we said earlier, here we have to take a *known undecidable* problem and convert it into the unknown one.

Example 1 (A_{TM} is undecidable). We know that $\text{HALT}_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM that halts on } w \}$ is undecidable. Let’s use that to prove that $A_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a TM that accepts } w \}$ is undecidable.

Solution. Before diving in, here’s the shape of what we’re about to do:

1. Build a new Turing machine M' whose behaviour depends on the original input.
2. Feed the encoding of M' to the decider we assumed exists, and read off the answer to the known undecidable problem.

We start with an input $\langle M, w \rangle$, and what we want to know is whether M halts on w . We’re going to show that if A_{TM} is decidable, then HALT_{TM} becomes decidable too.

First, build a new Turing machine M' that does the following when given an input x :

1. Run the machine M on input w , inside M' . (Think of a Turing machine as a big general-purpose computer — it can run other machines inside itself.)
2. If that run halts — whether M says accept or reject — then M' accepts x . If M halts on w , then after some finite amount of time it will definitely say accept or reject, since that's what halting means. Then M' says accept.

Look carefully at what's going on here. The input x is completely ignored — whether M' accepts x depends only on whether M halts on w . And if M never halts on w , then M' gets stuck too and loops forever on every input.

So let's see how M' behaves. If M halts on w , then no matter what you give M' , it always accepts — meaning $L(M') = \Sigma^*$. If M doesn't halt on w , then M' accepts nothing at all, so $L(M') = \emptyset$. The upshot: the mere fact that M' accepts *some* string is already a signal that M halts on w .

Now, for the sake of contradiction, assume A_{TM} is decidable and let R be the machine that decides it. Pick any string a you like and feed $\langle M', a \rangle$ to R . Two things can happen:

1. R says “Accept”. That means M' accepts a , which means that inside M' , the machine M halted on w . So we say accept for the halting problem.
2. R says “Reject”. That means M' can't accept a , which means M didn't halt on w . So we say reject for the halting problem.

Since R is a decider, this whole procedure always halts, and it decides $HALT_{\text{TM}}$. But we know that's impossible. So our assumption was wrong, and A_{TM} must be undecidable. $\square$

Example 2 (E_{TM} is undecidable). Now that we know A_{TM} is undecidable, let's use it. We'll prove that $E_{\text{TM}} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}$ is undecidable. This is the language that takes the encoding of a machine and accepts it when that machine accepts nothing at all.

Solution. We start with an input $\langle M, w \rangle$ and want to know whether M accepts w . We'll show that if E_{TM} is decidable, then A_{TM} becomes decidable. The approach is much like last time. Construct a Turing machine M' that, on input x , does the following:

1. Run the machine M on input w , inside M' .
2. If M accepts w , then M' accepts x . If M rejects w , then M' rejects x .

Once again x is ignored entirely — whether M' accepts depends only on what M does with w . If M accepts w , then M' accepts everything, so $L(M') = \Sigma^*$. If M doesn't accept w — either because it rejects, or because it loops forever — then M' accepts nothing, so $L(M') = \emptyset$. Putting it together:

$$L(M') \neq \emptyset \iff M \text{ accepts } w.$$

For the sake of contradiction, assume E_{TM} is decidable and let R be the machine deciding it. Feed $\langle M' \rangle$ to R . Look carefully here — we're handing over only the encoding of the machine, no input string, because that's what E_{TM} expects. Two things can happen:

1. R says “Reject”. That means $L(M')$ isn't empty, which means that inside M' , M accepted w . So we say accept for A_{TM} .
2. R says “Accept”. That means $L(M')$ is empty, which means M did not accept w . So we say reject for A_{TM} .

So we've decided A_{TM} , which is impossible. Hence E_{TM} is undecidable. $\square$